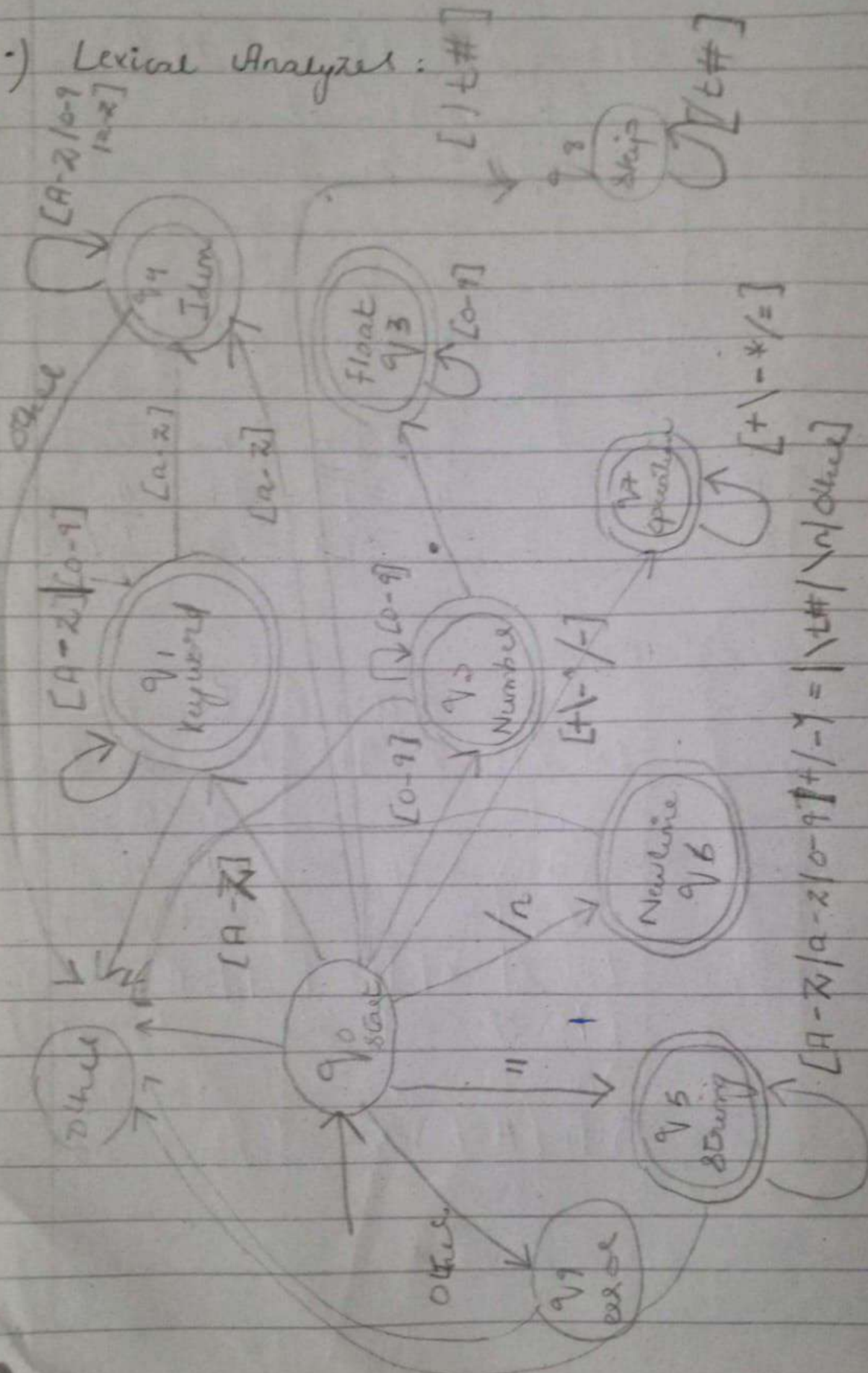


1.) Lexical Analyzer:



Transition table

	[A-Z]	[a-z]	[0-9]	["]	[+ - * / < > !]	[\\t#]	["\n"]	Other
q ₀	q ₁	q ₄	q ₂	q ₅	q ₇	q ₈	q ₆	q ₉
q ₁	q ₁	q ₄	q ₁	other	other	other	other	other
q ₂	other	other	q ₂	other	other	other	other	other
q ₃	other	other	q ₃	other	other	other	other	other
q ₄	q ₄	q ₄	q ₄	other	other	other	other	other
q ₅	other	other	other	other	other	other	other	other
q ₆	other	other	other	other	other	other	other	other
q ₇	"	"	"	other	q ₇	"	"	"
q ₈	"	"	"	"	other	q ₈	"	"

Regex Patterns

	Regex Pattern	Example
keyword	[A-Z][a-z 0-9]+	Forward, set
Number	\d+	80, 60, 6
Float	\d+ \. \d+	3.14, 1.5, 90.0
String	"[^\"]*"	"red", "blue"
Ident	[A-Z a-z -][A-Z a-z - 0-9]*	size, draw
Op	[+ - * / < > !]+	+, =, *, /
Newline	\n	(line break)
skip	[\\t]+ # [^\n]*	(ignored: space)

Parse Tree Example 01

Set size = 80

Proc draw len

PENDOWN

Forward len

Left 60

Forward len

Right 120

Forward len

Left 60

Pen up

End

Repeat 6

Call draw size

Right 60

End

Grammar rules used

program $\rightarrow$ statement $\#$

statement $\rightarrow$ set-stmt / proc-stmt / repeat-stmt

set-stmt $\rightarrow$ SET Ident '=' expr

proc-stmt $\rightarrow$ Proc Ident Ident body End

repeat-stmt $\rightarrow$ Repeat expr body end

body $\rightarrow$ statement $\#$

expr $\rightarrow$ Number / Float / Ident



Source Program # 2 :-

set radius = 50

color 'orange'

PENDOWN

if radius

forward radius

left 90

else

write 10

end

Pen up

Grammar use of

if-stmt $\Rightarrow$ if expr then-body [else else-body] end

then-body $\rightarrow$ statement *

else-body $\rightarrow$ statement *

cmd-stmt $\rightarrow$ keyword [expr]

expr $\rightarrow$ Number / Float / idenit / string

Program

Statement

set-stmt

set radius

=

expr

50

Statement

cmd-stmt

color

expr

strings ('orange')

Statement

cmd-stmt

pendown

Statement

if-stmt

expr

then-body

else

Statement

else-body

end

Statement

cmd-stmt

PENUP

cmd-stmt

forward radius

stmt

stmt

cmd-stmt

left 90

cmd-stmt

circle

10

expr

Semantic Analyzer:

scope 0 - Global
size Number 80

push on CALL draw

scope 1 Proc draw (local)
len Number 80 ← bound from 'size'

scope 1 - Proc draw (local)

len Number 80 ← bound from 'size'

Complete symbol Table - sample logs

Line	Name	Category	Type	Value/Binding	Scope
1	size	variable	Number	80	Global
2	draw	procedure	Proc	param = len, 8 stmts	Global
3	len	parameter	Number	bound at call time	scope(1) draw